inside the function definition is in principle perfectly OK — you are modifying the data pointed to, not the pointer itself.

"Isn't there some way to protect the data?" you may ask. Yes, there is: You can declare the *return type* of the subscript operator, `operator[]`, to be `const`. This is why there are two versions of `operator[]` in the `NRvector` class,

```
T & operator[](const int i);
const T & operator[](const int i) const;
```

The first form returns a reference to a modifiable vector element, while the second returns a nonmodifiable vector element (because the return type has a `const` in front).

But how does the compiler know which version to invoke when you just write `a[i]`? That is specified by the *trailing* word `const` in the second version. It refers not to the returned element, nor to the argument `i`, but to the object whose `operator[]` is being invoked, in our example the vector `a`. Taken together, the two versions say this to the compiler: "If the vector `a` is `const`, then transfer that const'ness to the returned element `a[i]`. If it isn't, then don't."

The remaining question is thus how the compiler determines whether `a` is `const`. In our example, where `a` is a function argument, it is trivial: The argument is either declared as `const` or else it isn't. In other contexts, `a` might be `const` because you originally declared it as such (and initialized it via constructor arguments), or because it is a `const` reference data member in some other object, or for some other, more arcane, reason.

As you can see, getting `const` to protect the data is a little complicated. Judging from the large number of matrix/vector libraries that follow this scheme, many people feel that the payoff is worthwhile. We urge you *always* to declare as `const` those objects and variables that are not intended to be modified. You do this both at the time an object is actually created and in the arguments of function declarations and definitions. You won't regret making a habit of `const` correctness.

In §1.4 we defined vector and matrix type names with trailing `_I` labels, for example, `VecDoub_I` and `MatInt_I`. The `_I`, which stands for "input to a function," means that the type is declared as `const`. (This is already done in the `typedef` statement; you don't have to repeat it.) The corresponding labels `_O` and `_IO` are to remind you when arguments are not just non-`const`, but will actually be modified by the function in whose argument list they appear.

Having rightly put all this emphasis on `const` correctness, duty compels us also to recognize the existence of an alternative philosophy, which is to stick with the more rudimentary view "`const` protects the container, not the contents." In this case you would want only *one* form of `operator[]`, namely

```
T & operator[](const int i) const;
```

It would be invoked whether your vector was passed by `const` reference or not. In both cases element `i` is returned as potentially modifiable. While we are opposed to this philosophically, it turns out that it does make possible a tricky kind of automatic type conversion that allows you to use your favorite vector and matrix classes instead of `NRvector` and `NRmatrix`, even if your classes use a syntax completely different from what we have assumed. For information on this very specialized application, see [1].

1.5.3 Abstract Base Class (ABC), or Template?

There is sometimes more than one good way to achieve some end in C++. Heck, let's be honest: There is *always* more than one way. Sometimes the differences amount to small tweaks, but at other times they embody very different views about the language. When we make one such choice, and you prefer another, you are going to be quite annoyed with us. Our defense against this is to avoid foolish consistencies,* and to illustrate as many viewpoints as possible.

A good example is the question of when to use an abstract base class (ABC) versus a template, when their capabilities overlap. Suppose we have a function `func` that can do its (useful) thing on, or using, several different types of objects, call them `ObjA`, `ObjB`, and `ObjC`. Moreover, `func` doesn't need to know much about the object it interacts with, only that it has some method `tellme`.

We could implement this setup as an abstract base class:

```
struct ObjABC {                          Abstract Base Class for objects with tellme.
    virtual void tellme() = 0;
};

struct ObjA : ObjABC {                   Derived class.
    ...
    void tellme() {...}
};
struct ObjB : ObjABC {                   Derived class.
    ...
    void tellme() {...}
};
struct ObjC : ObjABC {                   Derived class.
    ...
    void tellme() {...}
};

void func(ObjABC &x) {
    ...
    x.tellme();                          References the appropriate tellme.
}
```

On the other hand, using a template, we can write code for `func` without ever seeing (or even knowing the names of) the objects for which it is intended:

```
template<class T>
void func(T &x) {
    ...
    x.tellme();
}
```

That certainly seems easier! Is it better?

Maybe. A disadvantage of templates is that the template must be available to the compiler every time it encounters a call to `func`. This is because it actually compiles a different version of `func` for every different type of argument `T` that it encounters. Unless your code is so large that it cannot easily be compiled as a single unit, however, this is not much of a disadvantage. On the other side, favoring templates, is the fact that virtual functions incur a small run-time penalty when they are called. But this is rarely significant.

The deciding factors in this example relate to software engineering, not performance, and are hidden in the lines with ellipses (`...`). We haven't really told

*"A foolish consistency is the hobgoblin of little minds." —Emerson

you how closely related ObjA, ObjB, and ObjC are. If they are close, then the ABC approach offers possibilities for putting more than just tellme into the base class. Putting things into the base class, whether data or pure virtual methods, lets the compiler enforce consistency across the derived classes. If you later write another derived object ObjD, its consistency will also be enforced. For example, the compiler will require you to implement a method in every derived class corresponding to every pure virtual method in the base class.

By contrast, in the template approach, the only enforced consistency will be that the method tellme exists, and this will only be enforced at the point in the code where func is actually called with an ObjD argument (if such a point exists), not at the point where ObjD is defined. Consistency checking in the template approach is thus somewhat more haphazard.

Laid-back programmers will opt for templates. Up-tight programmers will opt for ABCs. We opt for... both, on different occasions. There can also be other reasons, having to do with subtle features of class derivation or of templates, for choosing one approach over the other. We will point these out as we encounter them in later chapters. For example, in Chapter 17 we define an abstract base class called StepperBase for the various "stepper" routines for solving ODEs. The derived classes implement particular stepping algorithms, and they are templated so they can accept function or functor arguments (see §1.3.3).

1.5.4 NaN and Floating Point Exceptions

We mentioned in §1.1.1 that the IEEE floating-point standard includes a representation for NaN, meaning "not a number." NaN is distinct from positive and negative infinity, as well as from every representable number. It can be both a blessing and a curse.

The blessing is that it can be useful to have a value that can be used with meanings like "don't process me" or "missing data" or "not yet initialized." To use NaN in this fashion, you need to be able to *set* variables to it, and you need to be able to *test* for its having been set.

Setting is easy. The "approved" method is to use numeric_limits. In nr3.h the line

```
static const Doub NaN = numeric_limits<Doub>::quiet_NaN();
```

defines a global value NaN, so that you can write things like

```
x = NaN;
```

at will. If you ever encounter a compiler that doesn't do this right (it's a pretty obscure corner of the standard library!), then try either

```
Uint proto_nan[2]=0xffffffff, 0x7fffffff;
double NaN = *( double* )proto_nan;
```

(which assumes little-endian behavior; cf. §1.1.1) or the self-explanatory

```
Doub NaN = sqrt(-1.);
```

which may, however, throw an immediate exception (see below) and thus not work for this purpose. But, one way or another, you can generally figure out how to get a NaN constant into your environment.

Testing also requires a bit of (one-time) experimentation: According to the IEEE standard, NaN is guaranteed to be the only value that is not equal to itself!

So, the "approved" method of testing whether Doub value x has been set to NaN is

```
if (x != x) {...}                It's a NaN!
```

(or test for equality to determine that it's not a NaN). Unfortunately, at time of writing, some otherwise perfectly good compilers don't do this right. Instead, they provide a macro isnan() that returns true if the argument is NaN, otherwise false. (Check carefully whether the required #include is math.h or float.h — it varies.)

What, then, is the *curse* of NaN? It is that some compilers, notably Microsoft, have poorly thought-out default behaviors in distinguishing *quiet NaNs* from *signalling NaNs*. Both kinds of NaNs are defined in the IEEE floating-point standard. Quiet NaNs are supposed to be for uses like those above: You can set them, test them, and propagate them by assignment, or even through other floating operations. In such uses they are not supposed to signal an exception that causes your program to abort. Signalling NaNs, on the other hand, are, as the name implies, supposed to signal exceptions. Signalling NaNs should be generated by invalid operations, such as the square root or logarithm of a negative number, or pow(0.,0.).

If all NaNs are treated as signalling exceptions, then you can't make use of them as we have suggested above. That's annoying, but OK. On the other hand, if all NaNs are treated as quiet (the Microsoft default at time of writing), then you will run long calculations only to find that all the results are NaN — and you have no way of locating the invalid operation that triggered the propagating cascade of (quiet) NaNs. That's *not* OK. It makes debugging a nightmare. (You can get the same disease if other floating-point exceptions propagate, for example overflow or division-by-zero.)

Tricks for specific compilers are not within our normal scope. But this one is so essential that we make it an "exception": If you are living on planet Microsoft, then the lines of code,

```
int cw = _controlfp(0,0);
cw &=~(EM_INVALID | EM_OVERFLOW | EM_ZERODIVIDE );
_controlfp(cw,MCW_EM);
```

at the beginning of your program will turn NaNs from invalid operations, overflows, and divides-by-zero into signalling NaNs, and leave all the other NaNs quiet. There is a compiler switch, _TURNONFPES_ in nr3.h that will do this for you automatically. (Further options are EM_UNDERFLOW, EM_INEXACT, and EM_DENORMAL, but we think these are best left quiet.)

1.5.5 Miscellany

- Bounds checking in vectors and matrices, that is, verifying that subscripts are in range, is expensive. It can easily double or triple the access time to subscripted elements. In their default configuration, the NRvector and NRmatrix classes never do bounds checking. However, nr3.h has a compiler switch, _CHECKBOUNDS_, that turns bounds checking on. This is implemented by preprocessor directives for conditional compilation so there is no performance penalty when you leave it turned off. This is ugly, but effective.

  The vector<> class in the C++ Standard Library takes a different tack. If you access a vector element by the syntax v[i], there is no bounds checking. If you instead use the at() method, as v.at(i), then bounds checking is performed. The obvious weakness of this approach is that you can't easily change a lengthy program from one method to the other, as you might want to

do when debugging.

- The importance to performance of avoiding unnecessary copying of large objects, such as vectors and matrices, cannot be overemphasized. As already mentioned, they should always be passed by reference in function arguments. But you also need to be careful about, or avoid completely, the use of functions whose return type is a large object. This is true even if the return type is a reference (which is a tricky business anyway). Our experience is that compilers often create temporary objects, using the copy constructor, when the need to do so is obscure or nonexistent. That is why we so frequently write `void` functions that have an argument of type (e.g.) `MatDoub_O` for returning the "answer." (When we do use vector or matrix return types, our excuse is either that the code is pedagogical, or that the overhead is negligible compared to some big calculation that has just been done.)

  You can check up on your compiler by instrumenting the vector and matrix classes: Add a static integer variable to the class definition, increment it within the copy constructor and assignment operator, and look at its value before and after operations that (you think) should not require any copies. You might be surprised.

- There are only two routines in *Numerical Recipes* that use three-dimensional arrays, `rlft3` in §12.6, and `solvde` in §18.3. The file `nr3.h` includes a rudimentary class for three-dimensional arrays, mainly to service these two routines. In many applications, a better way to proceed is to declare a vector of matrices, for example,

  ```
  vector<MatDoub> threedee(17);
  for (Int i=0;i<17;i++) threedee[i].resize(19,21);
  ```

  which creates, in effect, a three-dimensional array of size $17 \times 19 \times 21$. You can address individual components as `threedee[i][j][k]`.

- "Why no `namespace`?" Industrial-strength programmers will notice that, unlike the second edition, this third edition of *Numerical Recipes* does not shield its function and class names by a `NR::` namespace. Therefore, if you are so bold as to `#include` every single file in the book, you are dumping on the order of 500 names into the global namespace, definitely a bad idea!

  The explanation, quite simply, is that the vast majority of our users are not industrial-strength programmers, and most found the `NR::` namespace annoying and confusing. As we emphasized, strongly, in §1.0.1, NR is not a program library. If you want to create your own personal namespace for NR, please go ahead.

- In the distant past, *Numerical Recipes* included provisions for unit- or one-based, instead of zero-based, array indices. The last such version was published in 1992. Zero-based arrays have become so universally accepted that we no longer support any other option.

CITED REFERENCES AND FURTHER READING:

Numerical Recipes Software 2007, "Using Other Vector and Matrix Libraries," *Numerical Recipes Webnote No. 1*, at `http://www.nr.com/webnotes?1` [1]